

0.3. Flyttall

Datamaskinen drar et endelig antall bits (0/1) for å lagre tall.

Dette kan gi uønskettingheter!

F.eks. Beregn $0,1 + 0,1 + 0,1$ i Python!

Er vi heldige, forblir små feil små.

Men små feil i input kan også "eksplodere" til store feil i output!

(Bare å tenke på eksponentiell vekst)

Flyttall: Normaliserte reelle tall.

på formen $\pm \underbrace{1, \dots}_{0 \text{ eller } 1} \cdot 2^{\text{eksponent (heltall)}}$
mantisse m
 $1 \leq m < 2$

Eks: Hva er $n=23$ som flyttall?

$$\begin{aligned} 23 &\stackrel{1. \text{ binær}}{=} (10111)_2 \stackrel{2. \text{ normalisering}}{=} (1, 0111)_2 \cdot 2^4 \\ m &= \frac{23}{16} \end{aligned}$$

$(10111, 0)$
4

△

IEEE 754 Floating Point Standard

Maske representasjon av flyttall

Antall bits som brukes for å sette sammen tallet:

precision	sign	exponent	mantissa
single	1	8	23
double	1	11	52
long double	1	15	64



Maskin-representasjon (dobbel presisjon)

Flyttall med dobbel presisjon lagres med 64 bits i en datamaskin.

- 1 bit s til fortegnet.
 - ▶ $s = 0$ betyr positivt tall.
 - ▶ $s = 1$ betyr negativt tall.
- 11 bit til eksponenten e .
 - ▶ $-1022 \leq e \leq 1023$ → e kan variere mellom 1 og 2046.
 - ▶ Tallet $e + 1023$ lagres.
 - ▶ Eksponentbias $1023 = (011\ 1111\ 1111)_2$ trekkes fra for å få tilbake e .
- 52 bit til mantissa $1 \leq m < 2$.
 - ▶ Heltallet $M = 2^{52}(m - 1)$ lagres.
 - ▶ Mantissa regnes ut med $m = 1 + M \cdot 2^{-52}$.

Tallet 1 i dobbel presisjon

Vi skriver $1 = +1 \times 2^0$.

- Derfor er $s = 0$, $e = 0$ og $m = 1$.
- Vi legger til eksponentbias $1023 = (011\ 1111\ 1111)_2$ til eksponenten e .
- Heltallet $M = 2^{52}(m - 1) = 0$ lagres for mantissa.

Tallet 1 lagres derfor som

[0|011 1111 1111 | 0000 0000 0000 0000 0000
0000 0000 0000 0000 0000 0000 0000 0000]

= 3FF0 0000 0000 0000

på hexadesimal-form.

- 1) Merk at 2^{-53} er det største maskinrepresentative tallet mindre enn $\epsilon_M = 2^{-52}$ (machine epsilon) som oppfyller at flyttallet $1 + 2^{-53}$ er lik 1:

$$1 + \boxed{000000000000|0000000000000000\ldots00000000|1000\ldots} = 1$$

- 2) Merk også at $1 + \epsilon_M = 1 + 2^{-52}$ er det minste flyttallet større enn 1:

$$1 + \boxed{000000000000|0000000000000000\ldots00000001|0000\ldots} > 1$$

Machine epsilon ϵ_M

Mens $1 + 2^{-53}$ er samme flyttall som 1:
 $1 + [0000\ldots | 1000\ldots] = 1$

Avrunding i flyttallrepresentasjon:

- Husk: små feil i input kan gi store feil i output.
Derfor bryr vi oss om små avrundingsfeil.

x avrundes til det nærmeste flyttallet $fl(x)$

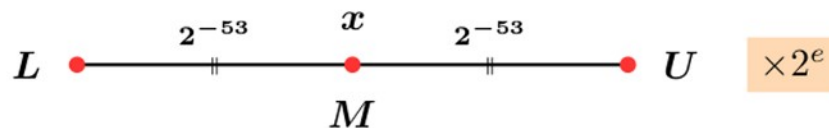
Ser på flyttallet og bit 53 osv. 3 muligheter:

- 0 bbbb (b er enten 0 eller 1) : dropp det
- 1 bbbb (ikke bare 0ere) : øk tallet



- 100000 (ligger akkurat mellom to tall i dobbelpres.)
unntakstilfelle
velger slik at siste siffer blir 0

Unntakstilfellet



Her er det definert en **unntaksregel** for hva $fl(x)$ skal være da begge alternativene $fl_L(x)$ og $fl_U(x)$ er jo like gode tilnærminger til denne x (som vist ovenfor):

Avrund slik at det siste sifferet (bit nr. 52 i mantissa til x) alltid blir 0.

Dette gjøres for å forsøke å hindre en uønsket (biased) effekt (oppbygning) av de akkumulerte avrundingsfeilene i beregninger.

Husk mentetall når en avrunder oppover!

Store og små tall

52 er det største positive heltallet k slik at $1 + 2^{-k}$ kan presenteres eksakt i dobbelpres. flyttall.

Men vi kan lage mindre tall:

$$(1 + 2^{-52}) \cdot 2^{-1022}$$

$$-1022 \leq e \leq 1023$$

Størst mulige tall får vi med $e = 1023$.

I praksis lagres tallet med $e + 1023 = 2026$.

EkspONENT 2047 brukes til $\pm \text{inf}$, hvis hele mantissen er lik 0

EkspONENT 0 brukes til å lagre enda mindre tall: og NaN, ellers

som ikke er normaliserte:

$$\pm 0, \text{p.b.b.b} \cdot 2^{-1022}$$

Minst mulig maskinrepresenterbar tall: $2^{-52} \cdot 2^{-1022} = 2^{-1074}$

Men: Å legge dette til 1 gjør ikke noe forskjell!

Hvis vi et flyttall i dobbelpres., så får vi det neste tallet ved å øke mantissen med 2^{-52} .

OBS: Selve tallet øker vi med $2^{-52} \cdot 2^e$
eksponent til x

Eks: Skriv om til flyttall $fl(x)$ og avrund etter IEEE regler.

a, $x = 1 + 2^{-53}$ Mantissa er $[0000 \dots 0000 / 1000 \dots]$
bare 0

unntakstilfelle 0 skal være 0

↳ dropper og får $fl(x) = 1$
2. normalisering

$$b, x = 0,1 = (0,00011)_2 = (1,1001)_2 \cdot 2^{-4}$$

3. Avrunding.

Mantisse er $10011001 \dots 1001$ | 1001

52

Øk tallet

Etter avrunding er mantissen

$10011001 \dots 10011010$

$$\text{Så } fl(0,1) = (1, \underbrace{1001}_{\times 12} 1001 \dots 1001 1010)_2$$

$$52 : 4 = 13$$

c) $x = -0,1$.

Dette har bare ulik fortegn fra $+0,1$, så

$$fl(-0,1) = -(1, \underbrace{1001}_{\times 12} 1001 \dots 1001 1010)_2$$

△

Føl. i avrundingen

$$x = (\pm 1, \underbrace{\dots}_m \cdot 2^e) \quad \text{til } fl(x).$$

$$\text{absolutt feil: } |fl(x) - x| \leq \frac{2^{-52}}{2} \cdot 2^e$$

↑
maks avstand i mantissen

$$|x| \geq 2^e \text{ gir}$$

$$\text{relativ feil: } \frac{|fl(x) - x|}{|x|} \leq \frac{\varepsilon_M}{2} \quad (\text{med } \varepsilon_M = 2^{-52} \text{ i Double pres.})$$

- 1) De spesielle tallene (**±Inf** og **NaN**) forsikrer at flyttallsaritmetikken gir et veldefinert flyttall og ikke vil føre til at programmet pr. default stopper opp eller kræsjer. Dette er viktig særlig for kritiske sanntidssimuleringer.
- 2) Også de spesielle verdier som returneres i unntakstilfeller er designet til å gi "korrekte" (gode) svar i de fleste tilfeller.
- 3) De små tallene (subnormal numbers) forsikrer også at for endelige flyttall x og y er $x-y = 0$ ekvivalent med $x = y$, som en kunne forvente, **men som ikke var tilfelle før IEEE 754.**
- 4) Tallene ovenfor er ikke nyttig kun for ekspertene, men forbedrer hverdagen for den jevne programmerer.

Vi går som sagt ikke inn på hvordan disse spesielle og små tallene er representert i IEEE 754 DP modellen.